

ngspice-46 vs. Spectre — Feature Gap Analysis

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

1

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

A capability comparison of the ngspice-46 build in this repository against a Spectre-class commercial simulator, to show where the open-source tool already matches the commercial one and where the gaps are. Grounded in a direct read of the ngspice-46/src/ tree (analyses, solver, devices, RF, convergence, parallelism), not marketing feature lists.

Legend: ✓ present / on par · △ partial, experimental, or via a workaround · ✗ absent. "Spectre" stands in for the commercial reference (Spectre / SpectreRF / Spectre X / RelXpert feature set).

The one column that matters is ngspice — Spectre has essentially everything, so its column is a baseline of ✓. The point of the table is where ngspice's mark is △ or ✗.

Context: the Verilog-A / OSDI device side is not a gap — thanks to the OpenVAF-reloaded compiler in this repository it is on par with (often ahead of) commercial Verilog-A support. The gaps below are all on the simulator/analysis side.

Core numerics

Category	Feature	ngspice	Spectre
Core solver	Sparse LU (KLU) direct solver	✓ ¹	✓
Core solver	Legacy Sparse1.3 fallback	✓	✓
Core solver	Parallel / partitioned / GPU linear solve	✗	✓
Core solver	Matrix reordering + scaling beyond KLU defaults	✓	✓
Integration	Trapezoidal + variable-order Gear	✓	✓
Integration	Advanced LTE-based step/order control	✓	✓
Convergence	gmin stepping + source stepping homotopy	✓	✓
Convergence	Pseudo-transient / dynamic-gmin continuation	✓	✓
Convergence	Damped / trust-region (globalized) Newton	✓	✓
Convergence	Coordinated accuracy presets (errpreset)	✓	✓

The "matrix reordering + scaling beyond KLU defaults" row is ✓ since [Enhancement-152](#): KLU's fill-reducing ordering (`klu_ordering=amd|colamd`), row scaling (`klu_scale=none|sum|max`), and BTF permutation (`klu_btf=on|off`) are now `.options` (previously hard-coded to AMD/max/on), and the broken `klu_memgrow_factor` is fixed. They change only how the matrix factors, not the solution.

The Integration "advanced LTE-based step/order control" row was △ because, although ngspice implements Gear coefficients for orders 1–6 (`NIcomCof`) and an LTE-limited-step estimate at any order (`CKTtrunc/CKTterr`), the stock transient controller in `dctran.c` only ever toggles the order between 1 and 2 — orders 3–6 were dead code on every ordinary run. [Enhancement-128](#) closes it with `.option dynorder`: each step it compares the raw (uncapped) LTE-limited step at the current order and its ± 1 neighbours and moves — with hysteresis, a settling hold, and an order-dependent growth cap — toward the order the local error rewards, so the higher orders are actually used. Off by default, bounded by `maxord`, verified under both linear solvers: 3–5× fewer timesteps at matched accuracy on a smooth RC decay and 8.9×

(and more accurate) on a smooth RLC ringdown, while a nonlinear switching circuit is left result-neutral to 5 significant figures.

These two convergence rows: ngspice's DC path is not naked — it has per-device junction limiting (30 device families), an optional nodedamping step clamp, and a multi-stage homotopy cascade (dynamic_gmin → new_gmin → spice3_gmin → source stepping). What it lacked vs. commercial tools was a principled globalized Newton (residual-based trust-region / Armijo) and a classic pseudo-transient ($\dot{X}$ -embedded) continuation — both now added. [Enhancement-111](#) adds the former: `.option linesearch`, an Armijo backtracking line search on a new KCL-residual merit $\|F\|=\|G \cdot x - b\|$ (result-neutral, off by default) — see the [implementation write-up](#). [Enhancement-153](#) completes this row with the trust-region counterpart: `.option trustregion`, a Levenberg-Marquardt Newton that damps the Jacobian $(x_{k+1}=x_k - (J+\mu I)^{-1}F, \mu=\lambda \cdot \|diag(J)\|, \text{Marquardt-scaled})$ rather than just the step length, re-aiming the step to regularize an ill-conditioned Jacobian — result-neutral, off by default, verified bit-identical to plain Newton. Its honest scope: because ngspice globalizes at the device level (junction limiting) before the residual is formed, a solver-level trust-region measures zero step-rejections on typical circuits and stays inert; it is a correct regularization for the residual-overshoot cases the device-level machinery does not catch. [Enhancement-112](#) then extended it to the KLU solver — it originally ran only under Sparse 1.3 and segfaulted under `.option klu` — so the line search now works under both linear solvers, with a residual-merit sequence numerically identical between them. [Enhancement-127](#) adds the latter: `.option ptcont`, a pseudo-transient continuation that embeds $f(x)=0$ in a backward-Euler pseudo-transient $f(x)+Gps \cdot (x-x_{prev})=0$ and marches the pseudo-timestep from small to large ($Gps \rightarrow 0$). The $Gps \cdot x_{prev}$ RHS coupling makes each step follow a stable trajectory (vs a static gmin step), so it converges — and to the physically correct root — on stiff circuits where plain Newton overshoots; result-neutral and off by default, verified under both linear solvers.

¹ KLU is compiled in (SuiteSparse, statically linked) but is not the default in this build — the default direct solver is Sparse 1.3; KLU is selected with `.option klu`. The two agree on DC / AC / transient, and — since [Enhancement-113](#) — on noise and single-ended pole-zero as well (the KLU adjoint solve was doing a non-transposed solve, silently wrong on asymmetric matrices; now fixed), and — since [Enhancement-114](#) — on DC/AC sensitivity (`.sens`), and — since [Enhancement-115](#) — on distortion (`.disto`) too. The only analysis still Sparse-only under KLU is balanced-output pole-zero; KLU is also less robust on stiff transient edges. Full behavior, defaults, and a solver-by-solver sweep of the example suite: [KLU vs. Sparse 1.3 solver notes](#).

Standard analyses (analog)

Category	Feature	ngspice	Spectre
Analyses	DC operating point / DC sweep	✓	✓
Analyses	Transient (<code>.tran</code>)	✓	✓
Analyses	AC small-signal (<code>.ac</code>)	✓	✓
Analyses	Noise, small-signal (<code>.noise</code> , onoise/inoise + integrated)	✓	✓
Analyses	Pole-zero (<code>.pz</code>)	✓	✓
Analyses	Transfer function (<code>.tf</code>)	✓	✓
Analyses	DC sensitivity (<code>.sens</code>)	✓	✓
Analyses	Distortion (<code>.disto</code>)	✓	✓
Analyses	Measurement / post-processing (<code>.meas</code>)	✓	✓

RF / periodic steady-state suite

Category	Feature	ngspice	Spectre
RF	S-parameter analysis + noise figure (<code>.sp</code>)	✓	✓
RF	Touchstone (<code>.sNp</code>) import / export	✓	✓
RF	Periodic steady state (PSS)	✓ ¹	✓
RF	Harmonic Balance (HB)	✓ ⁷	✓
RF	Periodic / phase noise (Pnoise)	✓ ³	✓
RF	Periodic AC (PAC, conversion gain)	✓ ²	✓
RF	Periodic transfer function (PXF)	✓ ⁴	✓
RF	Periodic S-parameters (PSP)	✓ ⁵	✓
RF	Quasi-periodic / multi-tone (QPSS / QPAC)	✓ ⁶	✓
RF	Envelope following	✓ ⁸	✓

¹ PSS was Δ (experimental `--enable-pss` flag, so `.pss` was unimplemented in shipped builds, and ~230 lines of shooting-loop trace per run). Since [Enhancement-117](#) it is built by default, quiet (trace behind `set ngdebug`), and verified against the analytic AC response; [Enhancement-118](#) then made it run under both linear solvers (KLU had hung on a timestep explosion from reused refactor pivots — now a full re-factor is forced each PSS step under KLU). It is still a brute-force shooting method and remains the foundation for the periodic small-signal analyses below. HB is now implemented -- see note 7 (it had shipped only as a `WITH_HB` stub).

² PAC is $\checkmark$ (complete periodic-AC analysis). The full chain is built and verified: [Enhancement-119](#) retains the periodic operating point, [Enhancement-120](#) extracts the periodic Jacobian harmonics G_k, C_k , [Enhancement-121](#) assembles and solves the $(2M+1)N$ harmonic conversion matrix $H_{\{nm\}} = G_{\{n-m\}} + j\omega_m \cdot C_{\{n-m\}}$, [Enhancement-122](#) wraps it in a user-facing `.pac` command (runs PSS, sweeps the input frequency, writes a complex plot), and [Enhancement-123](#) finishes it with a netlist-referenced small-signal source stimulus (a true transfer / conversion gain) and multi-sideband output — every conversion sideband $f_{in} + k \cdot f_0$ as its own named vector (`<node>_usb<k>`/`<node>_lsb<k>`). Verified on the RC: unit-current sideband-0 reproduces the AC driving-point impedance and, with `V1 AC 1`, sideband-0 reproduces the low-pass transfer $1/\sqrt{1+(2\pi fRC)^2}$ across the band while the conversion sidebands sit at floating-point zero (a linear circuit does not mix). The one scale caveat: the conversion matrix is solved densely (capped for modest circuits) on the brute-force shooting PSS. The same solve is the substrate for pnoise and PXF.

³ Pnoise is Δ (working, stationary-source first cut). [Enhancement-124](#) adds a `.pnoise` command that folds every device's noise through the adjoint of the conversion matrix ($H^T \Psi = e_{\{out,0\}}$): it loads the sideband- k transfer into `CKTrhs/CKTirhs` and calls the existing device noise routines (`NevalSrc`, `OSDI load_noise`) once per sideband, accumulating $\Sigma_k S \cdot |\Delta\Psi_k|^2$ — so it reuses every device noise model and needs no per-device code. Verified on the RC: because the circuit is linear the conversion matrix is block-diagonal, only sideband 0 contributes, and pnoise reduces exactly to `.noise` ($4kTR/(1+(2\pi fRC)^2)$, matching the `.noise` reference to every printed digit). [Enhancement-126](#) then adds a cyclostationary mode (`.pnoise ... cyclo`): it evaluates each device's noise PSD at every PSS sample's bias and folds it through the time-domain adjoint transfer, averaging over the period, so a pumped device's bias-dependent noise (a diode's $2qI(t)$, a resistor's flicker $\propto |I(t)|^2$) is captured correctly. It reduces exactly to the stationary case (and hence `.noise`) for a bias-independent source by Parseval, and on a flicker resistor carrying a known periodic current it gives $onoise \cdot f = R I^2 \cdot KF \cdot \langle I^2 \rangle$ using the period-average $\langle I^2 \rangle$ (matched to five digits). [Enhancement-140](#) closes the gap ($\checkmark$) with the oscillator phase-noise piece. `hbosc` is an autonomous harmonic balance: an oscillator has no source, so the HB residual $F(V) = I_R + [dq/dt] = 0$ is solved for the harmonics and the unknown oscillation frequency ω_0 , the singular conversion matrix (its phase mode $u_k = jk \quad v_k$) bordered with $dF/d\omega_0$ and a phase gauge and Newton-solved from a transient seed. `phasenoise` then reports $L(df)$: the adjoint of the conversion matrix at `OFFSET df`, unit at the carrier sideband, folds the device noise to the output; as $df \rightarrow 0$ the limit-cycle matrix goes singular through the phase mode, so the folded noise diverges as $1/df^2$ — the phase-noise skirt — normalized to the carrier power. Verified 8/8 on an LC oscillator: autonomous HB converges to f_0 and the describing-function amplitude, $L(df)$ has the -20 dB/dec skirt near the carrier flattening into the noise floor, at a physical absolute level, and the thermal noise scales as $L \propto T$ (doubling T raises L by exactly 3 dB).

^{*4} PXF is $\checkmark$ (complete). [Enhancement-125](#) adds a `.pxf` command — the adjoint of PAC. It solves $H^T \Psi = e_{\{out,0\}}$ per frequency and dots each sideband block of Ψ with the netlist AC-source pattern `B0` to get the input→output transfer at each sideband (`xf`, `xf_usb<k>`, `xf_lsb<k>`). By the identity $(H^{-1}B)_{out} = (H^{-T}e_{out})^T B$ the sideband-0 transfer is bit-identical to the PAC response at the output — a reciprocity cross-check that pins the adjoint solve — verified on the RC to equal the analytic low-pass transfer, with the conversion sidebands at floating-point zero. The same dense-solve scale caveat as PAC applies. This completes the PSS → PAC → Pnoise → PXF periodic small-signal suite.

⁵ PSP is $\checkmark$ since [Enhancement-132](#): a `.psp` command computes periodic small-signal S -parameters by exciting each RF port (`portnum/z0`, the `.sp` framework) through the same $(2M+1)N$ conversion matrix and forming $S^{\wedge}(k) = B^{\wedge}(k) \cdot A^{\wedge}-1$ per input frequency and conversion sideband $f_{in} + k \cdot f_0$. `pac_solve_at`'s matrix assembly is factored into a shared `pac_build_matrix`; the per-port solve drives each port's branch source ($V=1$) like `.sp`'s `VSRCspupdate`, and the power waves use the same Kurosawa convention, so $S = B \cdot A^{\wedge}-1$ is basis-invariant and reduces exactly to `.sp` for a time-invariant network. Verified 8/8: sideband-0 matches `.sp` to $\sim 10^{-16}$ for 1/2/3-port resistive and reactive networks (magnitude and phase) — including `OSDI Verilog-A` devices (conductance and reactive `ddt` stamps) — with correctly-zero conversion sidebands. Runs under both linear solvers (the conversion matrix is a standalone dense LU; PSS runs under both since E-118), verified bit-identical under KLU and Sparse.

⁶ QPSS is Δ (working two-tone, transient-sampling) since [Enhancement-133](#): a `qpss` command computes the two-tone steady-state spectrum — every mixing product $k1 \cdot f1 + k2 \cdot f2$ including IM3 — for commensurate tones (common beat $fb = \gcd(f1, f2)$). It runs an ordinary transient over a few beat periods, then evaluates the Fourier coefficient directly at each exact intermod frequency (no FFT-bin rounding) and labels it by the 2-D index $(k1, k2)$. Solver-independent (drives a transient), works with built-in and `OSDI` devices. Verified 7/7 checks incl. the analytic IM3 products and the 3:1 IP3 slope law. [Enhancement-136](#) then added the frequency-domain two-tone Harmonic Balance engine (`qpss <expr>`

`<f1> <f2> hb [K1] [K2]`): each node a 2-D Fourier series, devices sampled on a 2-D phase grid (θ_1, θ_2) (so incommensurate tones — irrational ratio, no common period — now work, which transient sampling cannot do) and 2-D DFT'd to the conversion matrix, Newton-solved by `pss_csolve` with the E-135 source stepping; sources captured by an oversampled least-squares APFT. Verified 7/7 (analytic IM3 ratio, the incommensurate $\sqrt{2}$ case, `HB==transient`, `KLU==Sparse`) and it retains its operating point. [Enhancement-137](#) closes the gap (✓) with small-signal QPAC — the two-tone analogue of PAC: `qpac <f_in>` injects a small signal around the retained QPSS operating point and reports the response at every sideband $f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$, mixing it through the same 2-D conversion matrix (`qp_build_matrix` at f_{in} , solved by `pss_csolve`) that the QPSS Newton used as its Jacobian. Verified 7/7, incl. `reduce-to-AC` (`pump→0` ? the direct response equals the plain `.ac` and the sidebands vanish) and the v^2 -pump conversion ratio. [Enhancement-138](#) adds QPnoise (`qpnoise <output_node> <f_in>`), the two-tone analogue of `pnoise` on the same operating point: one ADJOINT solve $H^T \Psi = e_{\{out, (0,0)\}}$ gives the transimpedance from every (node, sideband) to the output, and each device's `DEVnoise` folds $S \cdot |\Psi|^2$ over all sidebands (the mixer/PA noise conversion a static `.noise` cannot see). Verified 6/6, anchored by `reduce-to-noise` (`pump→0` ? `onoise = plain .noise = 4kTR`, exactly). [Enhancement-139](#) adds the cyclostationary mode (`qpnoise ... cyclo`): the device PSD $S(t)$ swings over the two-tone period, so instead of a single-bias fold it averages $(1/P) \cdot \sum_s S(t_s) \cdot |A_s|^2$ with A_s the inverse 2-D DFT of Ψ , re-biasing each phase sample (with per-sample junction settling). Reduces to the stationary case by Parseval when S is constant; a hard-pumped diode's switching shot noise makes `cyclo` $\sim 8\times$ the stationary estimate (10/10). [Enhancement-141](#) completes the two-tone small-signal suite with QPXF (`qpxf <output_node> <f_in>`), the ADJOINT of QPAC: one adjoint solve $H^T \Psi = e_{\{out, (0,0)\}}$, each sideband block of Ψ dotted with the AC-source pattern, gives the transfer from an input at every sideband $f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$ to the output. By the reciprocity identity the sideband-(0,0) transfer is bit-identical to the QPAC response (verified 6/6). So the quasi-periodic small-signal set now mirrors the single-tone PAC/Pnoise/PXF exactly: `QPSS → QPAC → QPnoise → QPXF`. [Enhancement-142](#) then gives all three a `dec|oct|lin` input-frequency sweep that emits a plottable `ngspice` plot (conversion gain / noise figure / image-rejection curves), matching how `.ac/.pnoise/.pxf` sweep — each swept point reuses the single-frequency solve and equals it to machine precision (5/5).

⁷ `HB` is ✓ since [Enhancement-134](#): a `hb <f0> <K>` command solves the periodic steady state in the frequency domain by Newton -- each node voltage a truncated Fourier series, the KCL residual $F_k = I_{R,k}(V) + [dq/dt]_k - I_{s,k} = 0$ driven to zero with the E-121 $(2K+1)N$ conversion matrix as the exact Jacobian. The device residual/Jacobian are sampled by driving DC+AC loads at the current iterate's voltages; nonlinear reactive elements need NO charge extraction because $dq/dt = C(v) \cdot v'$ -- the reactive current is the conversion matrix's `jwC` term applied to V , using the sampled $C(t)$. Solver-independent (`KLU + Sparse`): the dense complex Newton is `HB`'s own, so the linear solver is used only to read $G(t)/C(t)$ off the device matrix -- `hb_extract` carries the same `#ifdef KLU` complex-CSC binding as the PAC extraction, and `hb` honours `.option klu`, verified bit-identical under both. Built-in and OSDI devices. Verified 8/8 against the transient/fourier steady state, with quadratic Newton convergence: nonlinear `R` (analytic 3rd harmonic), nonlinear `R+C`, a real diode rectifier (junction devices are settled per sample so their limiter is a no-op), a compiled OSDI varactor whose $Q(v)$ 2nd harmonic matches, and `KLU==Sparse` parity. Strongly-driven circuits (where a cold full-strength Newton diverges) are handled by automatic source-stepping continuation ([Enhancement-135](#)): every source is ramped by λ : $0 \rightarrow 1$ in adaptive, warm-started, backtracking steps, so a 5 V diode rectifier that blows up cold converges in 3 continuation steps (easy circuits stay bit-identical). Single-tone; a sparse block solve and multi-tone `HB` (true incommensurate QPSS, cf. note 6) are the remaining follow-ups.

⁸ Envelope following is ✓ since [Enhancement-154](#): the `envelope` command computes the slow amplitude/phase envelope of a carrier-driven circuit by sampling the state once per carrier period $T=1/f_c$ and integrating the slow drift, jumping M periods at a time. The per-period map is $X_{\{n+1\}} = \text{phi}(X_n)$ (phi = one carrier period of DAE integration); the naive forward-Euler envelope jump is unstable on high- Q circuits (the one-period monodromy has unit-circle eigenvalues) — which is exactly why an earlier forward-Euler attempt blew up and was shelved. E-154 uses the implicit backward-Euler jump $X_{\{n+M\}} = X_n + M(\text{phi}(X_{\{n+M\}}) - X_{\{n+M\}})$, Newton-solved with the finite-difference monodromy $\text{Phi} = d\text{phi}/dY$ as $[(1+M)I - M \cdot \text{Phi}] dY = -G$; the A-stable step tracks a resonator's envelope without diverging and its fixed point is the true steady state. Step size M is chosen by step-doubling LTE control; the one-period map reuses the transient primitives on a fixed `nppp` grid in trapezoidal mode (backward-Euler damps high Q), self-started. Verified against a full `.tran` under both solvers: $<3\%$ across a $Q \sim 3160$ tank ring-up (26 envelope samples for ~ 3000 carrier periods), converging to the steady state, and $\sim 1.6\%$ on a $Q \sim 316$ tank. It pays off when the envelope is much slower than the carrier (high- Q / PLL / modulated-PA), where it is several times faster than the full transient; a second-order non-dissipative self-start and a sparse monodromy are follow-ups. With this, every analysis in the RF / periodic-steady-state suite is present.

Performance & scale

Category	Feature	ngspice	Spectre
Performance	Multithreaded device-model evaluation (OpenMP)	✓	✓

Category	Feature	ngspice	Spectre
Performance	Element bypass / latency exploitation	△	✓
Performance	Fast-SPICE hierarchical / isomorphic engine	×	✓
Performance	Distributed (MPI) / cloud partitioning	×	✓
Performance	Handles 10 ⁶ –10 ⁸ -node post-layout netlists	×	✓

Statistical / variability / yield

Category	Feature	ngspice	Spectre
Statistical	Monte Carlo	✓	✓
Statistical	Native process/mismatch modeling + correlations	✓	✓
Statistical	Low-discrepancy sampling (Sobol / Latin-hypercube)	✓	✓
Statistical	High-sigma methods (importance sampling, worst-case distance)	✓	✓
Statistical	Corner + MC + yield estimation flow	✓	✓

Monte Carlo was △(script-driven `alter + sgauss`); since [Enhancement-151](#) the `montecarlo` command packages the MC + spec + yield flow (Wilson-CI yield, optional LHS), so it is now ✓.

Low-discrepancy sampling is ✓ since [Enhancement-149](#): `mcsample lhs <N>` gives Latin-Hypercube stratification for the reset-driven `.param` Monte Carlo idiom (`agauss/gauss/aunif/unif/limit`), measured ~130× lower estimator variance at the same run count. Sobol and the `nutmeg-loop sgauss/sunif` idiom remain follow-ups.

High-sigma methods are ✓ since [Enhancement-150](#): `highsigma <N> -metric <expr> -max/-min <spec>` estimates 4–6 sigma rare-event failure probabilities by scaled-sigma importance sampling (inflate the Gaussian `.param` sigmas, reweight by the likelihood ratio) — direction-free, verified against the analytic $\Phi(-\beta)$ (e.g. a 5-sigma, 2.87e-7 event recovered from ~6000 runs). Mean-shift / worst-case-distance importance sampling remains a follow-up.

Process/mismatch correlations and the corner+MC+yield flow are ✓ since [Enhancement-151](#): `mccorr` registers a correlation matrix and `mvnorm(i)` draws correlated process/mismatch factors (composing with LHS and importance sampling), and `montecarlo` packages the spec-based yield estimate (Wilson CI, optional LHS); corners are the ordinary `.lib` selection. A matched divider demo yields ~100% process-correlated vs ~74% independent — the correlation model is what decides it.

Reliability / aging

Category	Feature	ngspice	Spectre
Reliability	Device aging (HCI / NBTI / TDDB)	×	✓
Reliability	Stress → degrade → re-simulate (fresh/aged) flow	×	✓
Reliability	Electromigration + IR-drop (EMIR)	×	✓

Post-layout / parasitics

Category	Feature	ngspice	Spectre
Post-layout	Flat parasitic (RC) netlist simulation	✓	✓
Post-layout	RC reduction / model-order reduction	×	✓
Post-layout	n-port (S/Y/Z) extracted-block import	✓	✓

Behavioral / mixed-signal / AMS

Category	Feature	ngspice	Spectre
Modeling	Verilog-A (analog, LRM Annex C) via OSDI	✓	✓
Modeling	XSPICE code models + event-driven digital	✓	✓
Modeling	Verilog-AMS mixed-signal (connect modules)	×	✓

Category	Feature	ngspice	Spectre
Modeling	Real-number modeling (wreal / RNM)	×	✓
Modeling	VHDL-AMS	×	✓

Interfaces & infrastructure

Category	Feature	ngspice	Spectre
Interface	Shared-library / programmatic API	✓	✓
Interface	Python bindings	✓	✓
Infrastructure	Built-in optimizer	✓	✓
Infrastructure	Checkpoint / restart of long runs	✓	✓

The “built-in optimizer” row is ✓ since [Enhancement-130](#): the `optimize` command is a derivative-free Nelder-Mead search that varies device/alter parameters, re-runs an analysis, and minimizes an objective expression in normalized $[0,1]$ space — verified to reach analytic optima in 1-D and 2-D. [Enhancement-143](#) adds a gradient-based least-squares mode: one or more weighted `-target <expr> <value>` measurements, optionally spread over several `-analysis` stages (so a single fit can combine, say, a DC operating point and an AC response), are fitted with Levenberg-Marquardt (finite-difference Jacobian) — the right tool for smooth curve-fitting / device-parameter-extraction problems, reaching the optimum in far fewer analysis runs than the simplex (e.g. 27 vs 67 evaluations on a two-target RC fit). Verified to recover both `is` and `n` of a compiled OSDI diode from two I-V points. [Enhancement-144](#) closes the last gap in the knob set: besides alter-reachable device/instance parameters (`-param`), the optimizer can now tune symbolic netlist `.param` values (`-dparam`) — since those are expanded at parse time, each candidate is applied with `alterparam` and a quiet reset that re-sources the deck (re-evaluating the `.param` expressions and re-stamping device values). Deck params are re-sourced first and the in-place alter params re-applied after, so the two kinds mix in one run; verified on `.params` used directly and inside expressions, including an OSDI-device value. [Enhancement-145](#) adds the third knob kind, `-mparam`, for `.model-card` parameters (named `@<model>[<param>]`): a model parameter is not alter-reachable (only sources, resistors and device instance parameters are — which is also exactly what `.dc` can sweep), so it is changed in place with `altermod` (no re-source, unlike `.param`). All three kinds — `-param` (instance/alter), `-mparam` (model/altermod), `-dparam` (`.param`/re-source) — mix in one optimization; verified recovering an OSDI and a built-in diode model parameter and a joint model+instance fit. [Enhancement-146](#) reuses the same knob-application machinery for a universal parametric sweep — a `sweep` command and `.sweep` card that step any knob (auto-detecting instance / model / `.param`) over a range, run a chosen inner analysis at each point and record the outputs into a plot. This is the parametric-analysis / `.step` capability commercial tools have: `.dc` can only step sources, resistors and instance parameters, whereas `sweep` also covers model parameters and symbolic `.params`. Verified to reproduce the built-in `.dc` bit-for-bit on an instance sweep and to match the analytic response on model-parameter, `.param`, AC and transient sweeps.

The “checkpoint / restart” row is ✓ since [Enhancement-131](#): stock ngspice could only continue a paused run in memory (`stop/resume`); the new `savestate <file> / loadstate <file>` commands serialize the full transient integration state (solution vector, device state history, time/step/order, pending breakpoints) to disk and resume it — including in a fresh process — so a long run survives a crash, splits across sessions, or moves between machines. The resumed waveform is bit-identical to an uninterrupted run for built-in devices (Sparse solver).

Where to invest (given the Verilog-A/OSDI side is done)

The realistic, winnable identity for this project is the open-source, OSDI/Verilog-A-native simulator with a real RF-noise and statistical story — not out-engineering 25 years of commercial fast-SPICE/parallel work. Ranked by leverage on the existing strength $\times$ differentiation $\times$ tractability:

1. RF periodic small-signal suite — PAC $\rightarrow$ Pnoise $\rightarrow$ PXF, on the hardened PSS. The PSS foundation is now shipped and verified under both linear solvers ([Enhancement-117](#), [Enhancement-118](#)), and [Enhancement-119](#) retains the periodic operating point (voltages + device states per sample) and [Enhancement-120](#) turns it into the periodic Jacobian harmonics `G_k`, `C_k`, and [Enhancement-121](#) assembles those into the $(2M+1)N$ harmonic conversion matrix and solves it, [Enhancement-122](#) wraps that in a working `.pac` command, and [Enhancement-123](#) finishes it with a netlist-referenced source stimulus and multi-sideband conversion-gain output — a complete periodic-AC analysis, verified against the exact linear transfer and driving-point responses, [Enhancement-124](#) adds `.pnoise` — folding every device’s noise through the conversion-matrix adjoint H^T (reusing the existing device noise routines), verified to reduce exactly to `.noise` in the linear limit — and [Enhancement-125](#)

adds `.pxf`, the adjoint transfer function, whose sideband-0 result is bit-identical to the PAC response by reciprocity. The PSS → PAC → Pnoise → PXF periodic small-signal suite is now complete — genuinely novel in open source — and [Enhancement-126](#) adds cyclostationary noise (per-sample bias, time-domain-transfer fold) so pumped devices' bias-dependent noise is handled correctly. The single-tone suite is now complemented by frequency-domain Harmonic Balance ([Enhancement-134](#)), oscillator phase noise ([Enhancement-140](#)), the two-tone quasi-periodic set (QPSS/QPAC/QPnoise/QPXF, [Enhancement-136–142](#)), and finally envelope following ([Enhancement-154](#), implicit monodromy period-jumping) — so every analysis in the RF / periodic-steady-state suite is now present. What remains is efficiency refinement (sparse conversion-matrix and monodromy solves, three-plus-tone) rather than new analyses.

2. Convergence robustness — coordinated accuracy presets (`errpreset`) landed in [Enhancement-110](#), a globalized Newton line search in [Enhancement-111/112](#), and pseudo-transient continuation (`.option ptcont`) in [Enhancement-127](#) — so the principled globalization and continuation gaps are now closed. What remains is mostly auto-triggering heuristics (reaching for these aids without the user asking) and folding them into the robustness presets. Self-contained in the ngspice core.
3. Statistical sampling — delivered. The whole statistical column is now ✓: Latin-Hypercube low-discrepancy sampling ([Enhancement-149](#), `mcsample lhs`, ~130× lower estimator variance), the high-sigma rare-event tail ([Enhancement-150](#), `highsigma` scaled-sigma importance sampling, 4–6 sigma from a few thousand runs), and native process/mismatch correlations plus a packaged yield flow ([Enhancement-151](#), `mccorr/mvnorm`
 - `montecarlo`). What remains is only efficiency variants (mean-shift / worst-case-distance importance sampling) and a one-command corner-sweep-of-yield wrapper.

Explicitly lower priority: fast-SPICE parallelism and aging/EMIR — enormous efforts that don't leverage what makes this project distinctive.